EBPF KERNEL DEFENSE • INDUSTRY-READY V1.0

AELFRA AEGIS

Comprehensive Master System Blueprint, Kernel Architecture, and Enterprise Production Deployment Guide

AUTHOR & LEAD ARCHITECT
Umar Ahmed

SECURITY FOCUS
Runtime Supply Chain Defense

INTERCEPTION ENGINE
Linux eBPF + BPF Ring Buffer

AUTONOMOUS ACTION
Sub-50ms Headless SIGKILL

Repository: github.com/mr-umar-ahmed/Aelfra-Aegis

Live Console: aelfra-aegis.vercel.app

1. Executive Summary & The Supply Chain Threat

Modern software engineering relies fundamentally on open-source package repositories like **npm** (JavaScript/TypeScript) and **PyPI** (Python). A standard enterprise application imports hundreds of transitive third-party dependencies.

When an engineer or automated CI/CD pipeline runs `npm install` or `pip install`, package managers automatically execute lifecycle scripts (such as `preinstall`, `install`, and `postinstall` in `package.json`, or `setup.py` in Python). **These scripts execute arbitrary code with full user or root privileges directly on the host or build runner.**

Why Traditional Static Scanners Fail

Industry-standard vulnerability tools (e.g. **Snyk, GitHub Dependabot, Trivy,** `npm audit`) are *static lockfile scanners*. They operate by checking known CVE databases (NVD) against package names in `package-lock.json`.

Threat Category	Static Scanners (Snyk, Dependabot)	Aelfra Aegis Runtime Guard
Zero-Day Malicious Packages	❌ **Blind** — No CVE exists in the database yet.	✅ **Catches Instantly** — Detects unauthorized `openat` / `execve`.
Typosquatting (`lodsh` vs `lodash`)	❌ **Blind** — Valid clean package name in lockfile.	✅ **Catches Instantly** — Flags child shells & edit-distance anomalies.
Account Takeover / Maintainer Hijack	❌ **Blind** — Legitimate version bump in registry.	✅ **Catches Instantly** — Flags credential access & C2 exfil sockets.
Obfuscated Dynamic Payloads	❌ **Blind** — Static AST cannot parse eval/base64 code.	✅ **Catches Instantly** — Observes true kernel-level execution.
Active Threat Mitigation	❌ **None** — Only generates post-build passive reports.	✅ **Automated SIGKILL** — Terminates malicious PIDs in < 50ms.

> **The Real-World Attack Scenario**
>
> An attacker publishes a typosquatted package `aegis-utils`. When installed, its `postinstall` hook reads `../../.env` to steal AWS keys, initiates an outbound HTTP POST to an external C2 server, and spawns `/bin/bash -c id` to test remote command execution. **Secrets are stolen in under 400 milliseconds before static tools can even scan the lockfile.**

2. What is Aelfra Aegis? (Our Solution)

Aelfra Aegis is an open-source, kernel-level runtime guard designed to detect, trace, and block software supply chain attacks in real-time. By leveraging **eBPF (Extended Berkeley Packet Filter)** probes attached directly inside the Linux kernel, Aegis intercepts critical system calls without modifying application code.

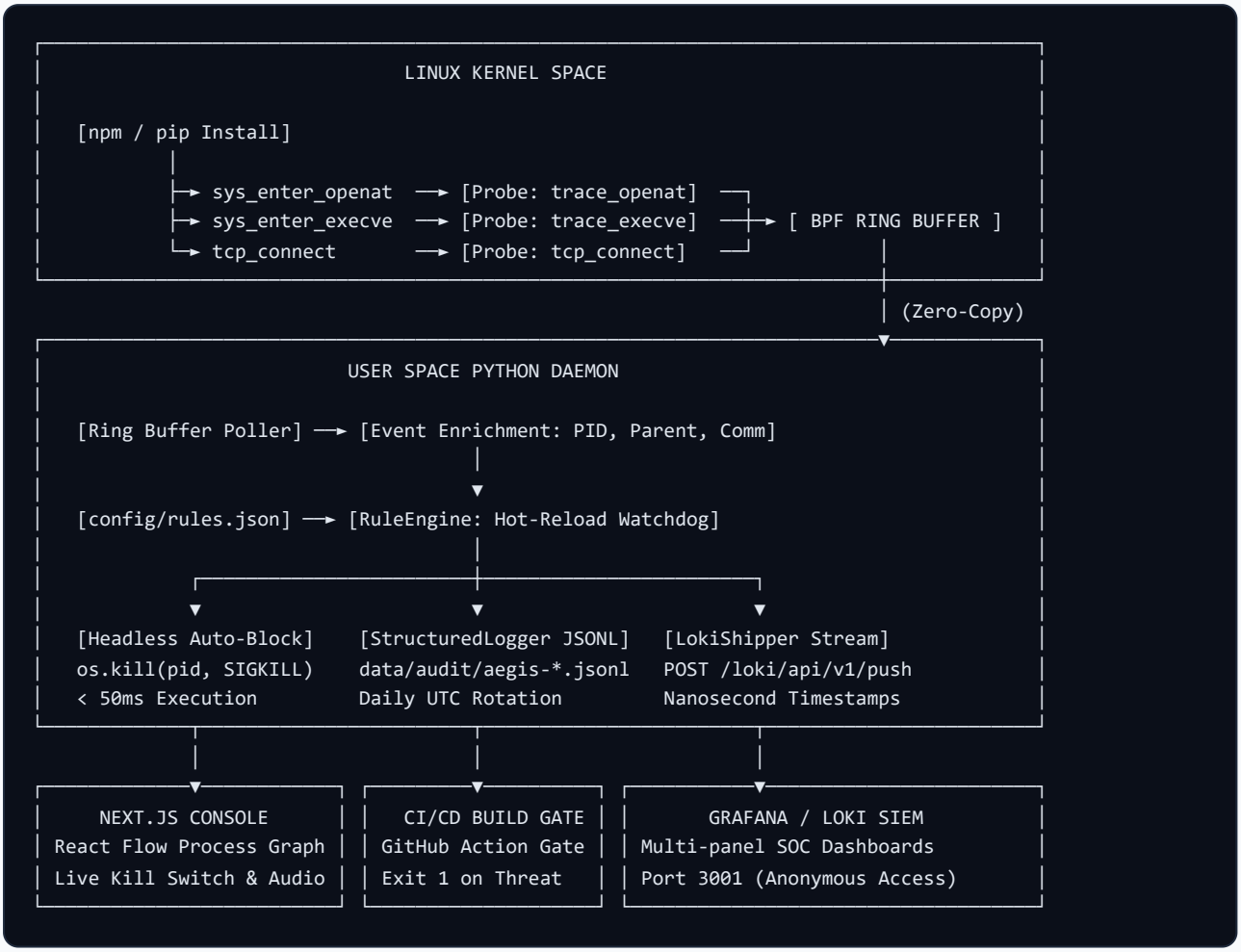
Core Pillars of Aegis Defense

- **Kernel-Level Interception:** Uses eBPF tracepoints and kprobes (`sys_enter_openat` , `sys_enter_execve` , `tcp_connect`) with **< 1% CPU overhead**.
- **BPF Ring Buffer Architecture:** Unified memory queue shared across all CPU cores with lockless, deterministic event ordering.
- **Hot-Reloading JSON Policy Engine:** Declarative rules (`config/rules.json`) mapped to **MITRE ATT&CK** techniques, auto-reloading every 5 seconds.
- **Autonomous Headless Blocking:** Sub-50ms automated `SIGKILL` execution for CI/CD pipelines without requiring human intervention.
- **Temporal Causal Provenance:** Interactive React Flow dashboard visualizing the full process tree (`npm` → `node` → `bash` → `network`).
- **SIEM-Compatible Telemetry:** Emits append-only JSON Lines (`.jsonl`) logs with daily UTC rotation and streams directly into **Grafana Loki**.
- **AI Threat Narration:** Built-in LLM integration delivering crisp 3-sentence plain-English intelligence summaries for SOC triage.

3. Complete Technology Stack & Specifications

Layer	Languages & Technologies	Purpose & Operational Function
Kernel Sensor	C, eBPF Bytecode, BPF Ring Buffer, Linux Tracepoints, kprobes, libbpf / CO-RE, BTF	Low-level kernel syscall interception, file descriptor reading, and event submission to user space.
User-Space Daemon	Python 3.11, BCC, ctypes, SQLite3, `socket`, `ipaddress`, `signal`, `argparse`	Polls ring buffer, enriches process metadata, evaluates JSON policy rules, and executes autonomous SIGKILL.
SIEM & Telemetry	JSON Lines (JSONL), Grafana Loki 2.9, Grafana 10.2, Docker Compose, LogQL	Unbuffered audit logging with daily UTC rotation and zero-cost local visual SIEM monitoring.
Web Console	Next.js 14 (App Router), TypeScript, React 18, `@xyflow/react`, TailwindCSS, Lucide Icons	Interactive temporal process graph, 1-tap mobile mode switcher, and live threat intelligence feed.
Threat Intelligence	WebSockets (`ws://8765`), Groq API (LLaMA 3.1 70B), AbuseIPDB API v2	Real-time bi-directional UI bridge, automated 3-sentence threat narratives, and C2 IP reputation scoring.
CI/CD & Testing	GitHub Actions, Docker, Shell Test Suite	Containerized dependency isolation, automated PR blocking gates, and log verification tools.

4. End-to-End System Architecture



5. Comprehensive Feature Breakdown

1. BPF Ring Buffer Kernel Probes

Hooks tracepoints for `openat` and `execve` along with `kprobe/tcp_connect`. Unlike legacy perf buffers which allocate per-CPU buffers leading to memory fragmentation, Aegis uses a single global ring buffer with deterministic event ordering.

2. Hot-Reloading Policy Rule Engine (`config/rules.json`)

Declarative JSON detection policies mapping directly to MITRE ATT&CK techniques:

- `CRED_001` (T1552.001): Credential File Access (`.env` , `.aws/credentials` , `id_rsa` , `secrets.json`).

- `EXEC_001` (T1059.004): Suspicious Child Execution from Package Manager (`npm` / `pip` spawning `bash` , `nc` , `curl`).
- `NET_001` (T1071.001): Unexpected Outbound Egress from Install Process on non-standard ports.
- `CHAIN_001` (T1020): Full Exfiltration Chain (File read + network connect within 30s for the same PID).

3. Three Operational Execution Modes

- `--mode=interactive` **(Default):** Starts the WebSocket bridge on `ws://0.0.0.0:8765` for live UI visualization.
- `--mode=headless` **(Autonomous Guard):** Automatically issues `SIGKILL` when confidence ≥ 90%. Writes reports to `data/incidents/` .
- `--mode=audit` **(Passive Compliance):** Never kills; writes structured logs to `data/audit/` and exits with non-zero codes in CI.

4. SIEM Structured Logger (`daemon/structured_logger.py`)

Emits append-only JSONL files (`data/audit/aegis-YYYY-MM-DD.jsonl`). Features zero-cron UTC daily rotation, unbuffered immediate flushing (`f.flush()`), and MITRE tactic resolution.

5. Grafana + Loki Monitoring Stack (`monitoring/`)

Pre-provisioned local Docker Compose stack hosting Loki (`:3100`) and Grafana (`:3001`) with pre-built dashboards for log volume, critical threats, and MITRE techniques.

6. Complete Step-by-Step Testing Guide

Track 1: In-Browser / Zero-Setup Testing (Any OS)

1. Open the live dashboard at **https://aelfra-aegis.vercel.app/**.
2. Complete the guided onboarding tour and enter the main console.
3. Click `SIMULATE ATTACK` → `Run Full Attack Chain` to observe live node spawning and threat narration.
4. Click the red `KILL [PID]` button on the compromised node to test process termination.

Track 2: Real Kernel Daemon & Attack Simulator (Linux / WSL2)

Open 3 separate terminals:

```
# Terminal 1: Start Mock C2 Exfil Listener
python3 simulator/listener.py

# Terminal 2: Start eBPF Security Daemon
sudo python3 daemon/daemon.py --mode=interactive

# Terminal 3: Trigger Real Malicious Package Attack
cd simulator/target-app
npm install --foreground-scripts
```

Track 3: Autonomous Headless Auto-Block Testing

```
# 1. Run in Headless Mode
sudo python3 daemon/daemon.py --mode=headless --threshold=90

# 2. Trigger Attack in another terminal
cd simulator/target-app && npm install --foreground-scripts

# 3. Verify Generated Incident Report
cat data/incidents/*.json
```

Track 4: Enterprise SIEM & Grafana/Loki Stack

```
# 1. Start Docker Stack
cd monitoring && docker compose up -d

# 2. Run Daemon with Loki Shipping
AEGIS_LOKI_URL=http://localhost:3100 sudo python3 daemon/daemon.py

# 3. Open Grafana at http://localhost:3001 (Anonymous Admin enabled)

# 4. Verify Log Hygiene
python3 scripts/verify-logs.py
```

Track 5: Standalone CLI Scanner & CI Gate

```
# Run Dependency Scan
python3 cli/aegis-scan.py package.json

# Run Automated Gate Test Suite
bash tests/test_ci_gate.sh
```

7. Technical Deep-Dives & FAQs

Q1: Why hook `sys_enter_openat` instead of `sys_enter_read`?

Hooking `read` produces hundreds of thousands of events per second, causing severe CPU degradation. Hooking `openat` inspects the file descriptor path at open time with < 1% CPU overhead, giving total security visibility with zero performance penalty.

Q2: What is CO-RE and why does production tooling use libbpf over BCC?

BCC requires Clang/LLVM compilers and kernel headers (~300MB) on target hosts. libbpf with CO-RE (Compile Once — Run Everywhere) compiles Ahead-Of-Time into a single portable `.bpf.o` binary that relocates struct offsets using kernel BTF metadata at runtime with zero target compiler dependencies.

Q3: How does Aegis prevent false positives on developers reading .env with vim or cat?

Rule `CRED_001` includes a `comm_not_in` whitelist (`["vim", "cat", "nano", "grep"]`), suppressing alerts for interactive user edits while aggressively flagging automated reads from package managers (`parent_comm_in: ["npm", "pip", "node"]`).